

1. Introduction to SOLID principles

SOLID principles are a set of five key object-oriented design guidelines that help developers build clean, scalable, and maintainable software. They were introduced by Robert C. Martin and are a foundational aspect of good software design. These principles encourage the creation of flexible, maintainable, and adaptable code that is easier to understand, extend, and test over time.

The acronym SOLID stands for:

1.1 Single Responsibility Principle (SRP)

- **Definition:** A class should have only one reason to change, meaning it should have a single responsibility or job to do.
- **Purpose:** The SRP promotes creating classes that do one thing and do it well. This makes the code more focused and easier to understand, test, and maintain.
- **Example (Violating SRP):** A Vehicle class that handles printing details, calculating value, and adding vehicles to a database violates SRP.
- **Example (Following SRP):** Separating the responsibilities into different classes like VehicleDetailPrinter, VehicleValueCalculator, and VehicleDatabaseManager adheres to SRP, where each class has a single, well-defined purpose.

1.2 Open-Closed Principle (OCP)

- **Definition:** Software entities should be open for extension but closed for modification, meaning you can add new behaviors without changing existing code.
- **Purpose:** OCP allows for easy addition of features without risking bugs in existing code.
- **Example:** Using interfaces allows adding new implementations (like different payment processors) without altering the class that uses them.

1.3 Liskov Substitution Principle (LSP)

- **Definition:** Subtypes should be substitutable for their base types without breaking the program's correctness.
- **Purpose:** LSP ensures that inheritance is used correctly and that derived classes fulfill the contract of the base class.
- **Example:** A subclass should not introduce constraints or behaviors that make it incompatible with where the base class is used.

1.4 Interface Segregation Principle (ISP)

- **Definition:** Clients shouldn't be forced to depend on interfaces they don't use; large interfaces should be broken down into smaller, specific ones.
- **Purpose:** ISP reduces coupling and makes code more modular by ensuring clients only depend on relevant methods.

- **Example:** Instead of one large interface for all vehicle types, create smaller interfaces for specific capabilities like Drivable or Flyable.

1.5 Dependency Inversion Principle (DIP)

- Definition: High-level modules and low-level modules should depend on abstractions (interfaces), not concrete implementations.
- Purpose: DIP decouples modules, improving flexibility, testability, and maintainability.
- Example: A class should depend on an interface (like Engine) rather than a specific implementation (like PetrolEngine), allowing for different engine types to be used easily.